Research Article

Alexandr Udeneev and Oleg Bakhteev

Learning Transferable Network Structure

https://doi.org/10.1515/sample-YYYY-XXXX
Received Month DD, YYYY; revised Month DD, YYYY; accepted Month DD, YYYY

Abstract: The connectivity and parameter-sharing structure of a neural network influences how well it generalizes from limited data and how quickly its numerical parameters can be optimized. We study how related tasks can reveal structural constraints that remain useful when numerical parameters are learned afresh. We represent these constraints by an indexator specifying permitted connections and groups of shared weights, with task-specific values supplied by a trainable codebook. Our central hypothesis is that structures extracted from source tasks can improve prediction on new tasks and help numerical optimization reach low-error solutions with less training. We investigate when such constraints are identifiable, how they can be extracted jointly from task solutions, and how their transfer affects prediction and convergence under matched training budgets.

Keywords: network structure learning, parameter sharing, multi-task learning, transfer learning

1 Introduction

Choosing an appropriate inductive bias is crucial for generalization [1]. By constraining the hypothesis space to reflect task structure, it can enable effective learning from limited data. Examples include local connectivity and spatial weight sharing in convolutional networks [2], and recurrent state updates with parameters shared across time [3]. Suitable parameter-sharing patterns also enforce equivariance to task-relevant transformations [4].

Selecting suitable structural biases by hand requires domain expertise, particularly when the relevant properties of a new task are not known in advance [5]. This motivates learning such biases from data. Yeh et al. [5] optimize parameter-sharing schemes to discover equivariances, while Zhou et al. [6] meta-learn sharing patterns across related tasks. Learning groups of similar weights has also been studied through soft weight tying [7] and weight-sharing regularization [8]. Atanov et al. [9] learn generative priors from convolutional filters of previously trained networks and use them to guide learning on new datasets. Schürholt et al. [10] learn generative representations of trained networks to sample new numerical weights. Sun et al. [11] extract task-specific subnetworks by iterative pruning and share parameters where these subnetworks overlap. Ando and Zhang [12] learn a shared predictive structure across tasks while allowing task-specific predictors.

More generally, a single task may admit multiple near-optimal structures, and its data alone need not identify which constraints will transfer to other tasks. Baxter [1] provides a theoretical framework for learning a shared hypothesis space across tasks, with generalization bounds for new tasks from the same environment under suitable capacity and sampling assumptions. Our focus is on discovering connectivity and weight-sharing constraints that remain useful across tasks, with numerical values learned from scratch for each new task.

In this paper, we study multi-task structure learning through the extraction of shared constraints from solutions to related tasks. We represent a structure by an indexator that specifies which connections are present and which weights share a numerical value. The corresponding codebook contains the task-specific trainable values, while all remaining numerical parameters can adapt freely to each task. We assess candidate structures by their validation performance after fresh numerical training on source tasks, and examine whether the selected constraints remain useful on held-out tasks.

Our central hypothesis is that structural regularities shared across source solutions can improve both generalization and the progress of numerical optimization on related tasks. Once a structure has been selected, its connectivity and sharing constraints are held fixed while its codebook and remaining numerical parameters are initialized and learned anew on each target task. This allows us to assess prediction quality at a fixed training budget and the amount of training needed to reach a prescribed error level. The cost of extracting a structure from source tasks is reported separately from the cost of learning numerical parameters on target tasks.

Our contributions are as follows:

– We investigate the identifiability of common connectivity and parameter-sharing constraints across related tasks, accounting for task-specific numerical parameters and functional equivalences. (in progress)
– We study methods for extracting transferable indexators by combining structural information from source-task solutions. (in progress)
– We evaluate the effect of extracted structures on prediction quality and numerical convergence on new tasks after fresh parameter initialization, comparing with baselines under common data and numerical adaptation protocols. (in progress)

2 Related Works

Learning a shared hypothesis space from related tasks is a classical formulation of inductive-bias learning [1]. For neural networks, structural constraints can express locality, equivariance and parameter sharing [4]. Deep Sets [13] provides a useful example of explicitly encoded invariance, whereas here the relevant connections and equality groups are not supplied to the learner.

The closest structural competitors parameterize weights as a shared linear map applied to task-specific coefficients. Yeh et al. [5] learn an assignment matrix using a differentiable relaxation and an implicit hypergradient. MSR [6] meta-learns a continuous reparameterization, together with numerical initialization and adaptation rates. We compare native continuous representations separately from hard indexators, and distinguish transfer of a structural constraint from transfer of learned numerical initial values.

Weight tying and quantization also learn equality groups. Soft weight tying [7], trained quantization [14], LUT-Q [15] and weight-sharing regularization [8] establish that learning a dictionary and assignments is not itself new. Our experimental question is whether a common assignment extracted from several source solutions remains useful after replacing every task-specific numerical value. The proposed component is joint calibrated projection of aligned solutions, followed by functional comparison of a finite bank; the dictionary representation and the general principle of learning structure across tasks are prior art.

3 Preliminary

Related prediction tasks may benefit from the same relations between weights even when they require different numerical values. For example, a group of connections may share a coefficient in each task, while its value changes from one task to another. Reusing this pattern constrains the parameters learned on a new task. Such constraints can help a predictor generalize from limited data and change which solutions an optimizer finds within a given budget. We study this form of transfer by separating the structure of a network from its numerical parameters. The structure is proposed from experience with previous tasks; its usefulness is judged after the numerical parameters have been learned anew.

3.1 Multi-task learning

Multi-task learning uses regularities shared by related tasks to improve learning. Here, each task τ is a supervised prediction problem with its own distribution P_τ over input–target pairs $(\mathbf{x}, y)$. Tasks belong to a family $\mathcal{T}$, share input and target spaces $\mathcal{X}$ and $\mathcal{Y}$, and occur according to a distribution $p(\tau)$ [1]. We consider task families in which structural patterns useful for solving one task may also help solve others.

All tasks in the source collection $\mathcal{T}_{\text{tr}}$ and the target collection $\mathcal{T}_{\text{val}}$ are sampled independently from $p(\tau)$. We learn shared patterns from the source tasks and assess their transfer on the target tasks. For each task, a *support sample* S_τ^{s} is used to fit numerical parameters, and an independent *query sample* S_τ^{q} is used to compare the fitted predictors [6, 16]. Conditional on τ, both samples consist of independent draws from P_τ. For a predictor f and a sample S, the empirical loss is

$$\mathcal{L}_S(f) := \frac{1}{|S|} \sum_{(\mathbf{x},y)\in S} \ell(f(\mathbf{x}), y). \tag{1}$$

Here, ℓ measures prediction error on one example, such as squared error for regression or cross-entropy for classification.

3.2 Network structure

We distinguish structural constraints shared across tasks from task-specific numerical values. A network structure specifies which connections are permitted and which weights share a trainable parameter. Let P denote the number of weights subject to these constraints, and let $\mathbf{w}_\tau \in \mathbb{R}^P$ collect them in a fixed order on task τ, across any number of layers.

We encode the constraints using an *indexator*, an integer vector $\mathbf{I} \in \{0, \dots, K\}^P$, where K is a prescribed upper bound on the number of groups with independently trainable values. Their task-specific values form a *codebook* $\mathbf{C}_\tau = (C_{\tau,0}, \dots, C_{\tau,K}) \in \mathbb{R}^{K+1}$ [5, 14]. The weights are obtained by the lookup rule

$$w_{\tau,p} = C_{\tau,I_p}, \qquad p = 1, \dots, P, \qquad C_{\tau,0} = 0. \tag{2}$$

Under this rule, $I_p = 0$ removes a connection, while $I_p = I_q > 0$ requires the corresponding weights to remain equal throughout training. The indexator therefore determines both connectivity and the partition of permitted connections into parameter-sharing groups. Allowing the codebook values to vary defines the associated space of admissible weights,

$$\mathcal{W}_{\mathbf{I}} := \left\{ (C_{I_1}, \dots, C_{I_P})^{\mathsf{T}} : \mathbf{C} \in \mathbb{R}^{K+1},\ C_0 = 0 \right\}. \tag{3}$$

Positive labels merely name the groups, so consistently renaming them leaves this space unchanged. Distinct groups have independent parameters even if their learned values happen to coincide on a task.

We collect all remaining unconstrained trainable parameters in $\boldsymbol{\theta}_\tau$ and write $f_{\mathbf{I},\mathbf{C}_\tau,\boldsymbol{\theta}_\tau}$ for the resulting predictor. Reusing $\mathbf{I}$ across tasks preserves the constraints defining $\mathcal{W}_{\mathbf{I}}$, while each task learns its own codebook and remaining numerical parameters. In every fitting attempt, the indexator is held fixed, and all trainable entries of $\mathbf{C}_\tau$ and $\boldsymbol{\theta}_\tau$ are initialized afresh and optimized on the task's support sample.

3.3 Problem formulation

Let $\mathcal{I}_K := \{0, \dots, K\}^P$ denote the set of admissible indexators. For each $\mathbf{I} \in \mathcal{I}_K$, let $\mathcal{F}_{\mathbf{I}}$ denote the set of predictors $f_{\mathbf{I},\mathbf{C},\boldsymbol{\theta}}$ obtained as the admissible numerical parameters $\mathbf{C}$ and $\boldsymbol{\theta}$ vary.

Definition 1 (Functional equivalence of indexators)**.** Two indexators $\mathbf{I}, \mathbf{J} \in \mathcal{I}_K$ are functionally equivalent, written $\mathbf{I} \sim \mathbf{J}$, if $\mathcal{F}_{\mathbf{I}} = \mathcal{F}_{\mathbf{J}}$. We write $[\mathbf{I}]$ for the equivalence class of $\mathbf{I}$ and $\mathcal{I}_K/\!\sim$ for the set of these classes.

For a task τ and a fixed indexator $\mathbf{I} \in \mathcal{I}_K$, define the optimal task risk as

$$\mathcal{R}_\tau^\star(\mathbf{I}) := \inf_{\mathbf{C}_\tau, \boldsymbol{\theta}_\tau} \mathbb{E}_{(\mathbf{x},y)\sim P_\tau} \left[\ell\big(f_{\mathbf{I},\mathbf{C}_\tau,\boldsymbol{\theta}_\tau}(\mathbf{x}), y\big)\right]. \tag{4}$$

The infimum optimizes task-specific numerical parameters with respect to the true task distribution, giving an ideal measure of the predictive capacity of $\mathbf{I}$. Generalization from finite support samples and numerical convergence also depend on the fitting procedure. Equivalent indexators have the same optimal task risk, so this criterion is independent of the representative within each class. The population problem can therefore be written as a search over equivalence classes:

$$[\mathbf{I}^\star] \in \operatorname*{arg\,min}_{[\mathbf{I}]\in\mathcal{I}_K/\sim} \mathbb{E}_{\tau\sim p(\tau)} \left[\mathcal{R}_\tau^\star(\mathbf{I})\right]. \tag{5}$$

The expectation concerns a new task drawn from the same task environment, and any representative of $[\mathbf{I}]$ can be used to evaluate the objective. Structure learning is thus defined up to functional equivalence. Distinct equivalence classes may still attain the same optimum; identifying a unique optimal class requires suitable assumptions on the task family and network parameterization.

For the empirical problem, the indexator is selected on source tasks and then held fixed when fitting numerical parameters on held-out tasks. The resulting numerical adaptation problem is

$$\begin{aligned} &\inf_{\{\mathbf{C}_\tau,\boldsymbol{\theta}_\tau\}_{\tau\in\mathcal{T}_{\text{val}}}} \frac{1}{|\mathcal{T}_{\text{val}}|} \sum_{\tau\in\mathcal{T}_{\text{val}}} \mathcal{L}_{S_\tau^{\text{s}}}\big(f_{\widehat{\mathbf{I}},\mathbf{C}_\tau,\boldsymbol{\theta}_\tau}\big) \\ &\text{subject to} \quad \widehat{\mathbf{I}} \in \operatorname*{arg\,min}_{\mathbf{I}\in\mathcal{I}_K} \frac{1}{|\mathcal{T}_{\text{tr}}|} \sum_{\tau\in\mathcal{T}_{\text{tr}}} \inf_{\mathbf{C}_\tau,\boldsymbol{\theta}_\tau} \mathcal{L}_{S_\tau^{\text{s}}}\big(f_{\mathbf{I},\mathbf{C}_\tau,\boldsymbol{\theta}_\tau}\big). \end{aligned} \tag{6}$$

Transfer performance is measured by the mean query loss across held-out tasks after their numerical parameters have been fitted on the support samples.

4 Method

We extract a shared indexator by jointly projecting numerical solutions of source tasks and comparing the resulting structures after numerical refitting. Projection proposes groups; prediction error selects the structure.

4.1 Source solutions and their alignment

Write $T := |\mathcal{T}_{\text{tr}}|$ and $\boldsymbol{\eta}_\tau := (C_{\tau,1}, \ldots, C_{\tau,K}, \boldsymbol{\theta}_\tau)$ for the trainable numerical parameters. For each source task, fitting uses S_τ^{s}. Its query sample is partitioned into V_τ^{num}, used for numerical checkpoints and proposal construction, and V_τ^{str}, reserved for the final structure comparison. Target tasks do not enter this procedure.

First, unconstrained numerical fitting gives $\widetilde{f}_\tau = f_{\widetilde{\mathbf{w}}_\tau,\widetilde{\boldsymbol{\theta}}_\tau}$, with $\widetilde{\mathbf{w}}_\tau \in \mathbb{R}^P$. Dense starts are fitted on support data and selected on V_τ^{num}. The numerical updates are specified below; for dense fitting their variables are the unrestricted network parameters.

Definition 2 (Source descriptor)**.** For a fixed alignment of a source solution, its descriptor is

$$\mathbf{z}_\tau(\mathbf{a}_\tau) = \mathbf{B}_\tau \mathbf{a}_\tau + \mathbf{v}_\tau, \qquad \mathbf{u}_\tau(\mathbf{a}_\tau) \in (0,\infty)^P, \tag{7}$$

where $\mathbf{a}_\tau \in \mathbb{R}^{r_\tau}$ contains continuous calibration variables, $\mathbf{B}_\tau \in \mathbb{R}^{P\times r_\tau}$ and $\mathbf{v}_\tau \in \mathbb{R}^P$ are obtained from the source solution, and $\mathbf{u}_\tau$ assigns positive discrepancy scales.

The identity descriptor has $r_\tau = 0$, $\mathbf{v}_\tau = \widetilde{\mathbf{w}}_\tau$ and $\mathbf{u}_\tau = \mathbf{1}$, and is available for any network. Architecture-specific descriptors can account for parameter symmetries. Discrete alignments, such as hidden-unit permutations, are fixed for each projection run; continuous calibration is updated within the run. Adjacent parameters must be transformed together when an alignment is claimed to preserve the predictor.

4.2 Joint discrete projection

For a binary support $\mathbf{M} \in \{0,1\}^P$, let $\mathcal{I}_K(\mathbf{M}) := \{\mathbf{I} \in \mathcal{I}_K : I_p = 0 \Leftrightarrow M_p = 0 \text{ for all } p\}$. With $\mathbf{c}_\tau := (C_{\tau,1}, \dots, C_{\tau,K})^\mathsf{T}$, define the lookup matrix by

$$(\mathbf{A}_\mathbf{I})_{pj} := \mathbf{1}\{I_p = j\}, \qquad \mathbf{w}_\tau = \mathbf{A}_\mathbf{I}\mathbf{c}_\tau. \tag{8}$$

We score a joint projection by

$$\begin{gathered}\mathcal{Q}(\mathbf{I}, \{\mathbf{c}_\tau, \mathbf{a}_\tau\}) := \frac{1}{TP}\sum_{\tau,p} \rho_\nu\left(\frac{(\mathbf{A}_\mathbf{I}\mathbf{c}_\tau)_p - z_{\tau,p}(\mathbf{a}_\tau)}{u_{\tau,p}(\mathbf{a}_\tau)}\right), \\ \rho_\nu(v) := \nu \log(1 + v^2/\nu), \qquad \nu > 0.\end{gathered} \tag{9}$$

Assignments are shared across tasks; code values and calibrations are task-specific. This tests whether positions admit common values *within* each task.

With $\mathbf{I}$ and the discrepancy scales fixed, calibration is a constrained weighted least-squares problem. Set

$$\begin{gathered}\boldsymbol{\xi}_\tau := (\mathbf{c}_\tau^\mathsf{T}, \mathbf{a}_\tau^\mathsf{T})^\mathsf{T}, \qquad \mathbf{E}_{\tau,\mathbf{I}} := [\mathbf{A}_\mathbf{I}, -\mathbf{B}_\tau], \\ \boldsymbol{\xi}_\tau^+ \in \operatorname*{arg\,min}_{\mathbf{G}_{\tau,\mathbf{I}}\boldsymbol{\xi} = \mathbf{h}_{\tau,\mathbf{I}}} \left\|\mathbf{D}_\tau^{-1}(\mathbf{E}_{\tau,\mathbf{I}}\boldsymbol{\xi} - \mathbf{v}_\tau)\right\|_2^2, \qquad \mathbf{D}_\tau := \operatorname{diag} \mathbf{u}_\tau(\mathbf{a}_\tau).\end{gathered} \tag{10}$$

The feasible affine constraints $\mathbf{G}_{\tau,\mathbf{I}}\boldsymbol{\xi} = \mathbf{h}_{\tau,\mathbf{I}}$ fix calibration scale freedoms; they are absent for the identity descriptor. A minimum-norm minimizer is used when the solution is nonunique. Equivalently, calibration solves

$$\begin{pmatrix} \mathbf{E}^\mathsf{T}\mathbf{D}^{-2}\mathbf{E} & \mathbf{G}^\mathsf{T} \\ \mathbf{G} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \boldsymbol{\xi}^+ \\ \boldsymbol{\lambda} \end{pmatrix} = \begin{pmatrix} \mathbf{E}^\mathsf{T}\mathbf{D}^{-2}\mathbf{v} \\ \mathbf{h} \end{pmatrix}, \tag{11}$$

with task and indexator subscripts suppressed. The scales are recomputed between solves. This least-squares update is a surrogate for the robust score; monotone decrease of $\mathcal{Q}$ is not assumed.

Algorithm 1: Joint calibration and shared assignments

Input: Aligned descriptors; support $\mathbf{M}$; $\mathbf{I}^{(0)} \in \mathcal{I}_K(\mathbf{M})$; initial calibrations; budgets $N_{\text{proj}}, N_{\text{cal}}$.
Output: A proposed indexator and its projection score.

$\mathbf{I} \leftarrow \mathbf{I}^{(0)}$
for $k = 0, \dots, N_{\text{proj}}$ **do**
  foreach $\tau \in \mathcal{T}_{\text{tr}}$ **do**
    for $q = 1, \dots, N_{\text{cal}}$ **do**
      $\mathbf{D}_\tau \leftarrow \operatorname{diag} \mathbf{u}_\tau(\mathbf{a}_\tau)$
      $\mathbf{E}_{\tau,\mathbf{I}} \leftarrow [\mathbf{A}_\mathbf{I}, -\mathbf{B}_\tau]$
      $\boldsymbol{\xi}_\tau \leftarrow$ solution of (11)
  if $k = N_{\text{proj}}$ **then**
    break
  foreach p *with* $M_p = 1$ *and* $j = 1, \dots, K$ **do**
    $d_{pj} \leftarrow \sum_\tau \rho_\nu\left(\frac{C_{\tau,j} - z_{\tau,p}(\mathbf{a}_\tau)}{u_{\tau,p}(\mathbf{a}_\tau)}\right)$
  `// The same label must fit every source task.`
  $I_p^+ \leftarrow 0$ if $M_p = 0$; otherwise $I_p^+ \in \arg\min_{j=1,\dots,K} d_{pj}$ $(p = 1, \dots, P)$
  if $\mathbf{I}^+ = \mathbf{I}$ **then**
    break
  $\mathbf{I} \leftarrow \mathbf{I}^+$
return $(\mathbf{I}, \mathcal{Q}(\mathbf{I}, \{\mathbf{c}_\tau, \mathbf{a}_\tau\}))$

Assignment ties use a fixed ordering of labels. Single-source runs of Algorithm 1 provide initial structures. After numerical fitting across tasks, a subset is retained as anchors $\mathcal{A}$. For each anchor alignment, alternative

supports include its own mask and source-consensus masks

$$M_p^{(\lambda,v)} := \mathbf{1}\left\{\sum_{\tau} \mathbf{1}\left\{|z_{\tau,p}(\mathbf{a}_\tau^{(0)})| > \max\{\lambda u_{\tau,p}(\mathbf{a}_\tau^{(0)}), \varepsilon\}\right\} \geq v\right\}, \tag{12}$$

where $\varepsilon > 0$ and the finite sets of thresholds λ and vote counts $v \in \{1, \dots, T\}$ are prescribed. Joint runs start from anchors, single-source assignments and random active labels.

Let $\mathcal{G}$ be a finite group of known admissible label and parameter permutations. We identify duplicates by the canonical form $\kappa(\mathbf{I}) := \min^{\text{lex}}_{g\in\mathcal{G}} g\mathbf{I}$. The bank $\mathcal{B}$ contains the best canonically distinct proposals ranked by $\mathcal{Q}$, together with $\kappa(\mathcal{A})$. This removes known symmetries without requiring a test for all functional equivalences.

4.3 Numerical refitting and functional selection

For a fixed candidate, write $F_{\tau,\mathbf{I}}(\boldsymbol{\eta}) := \mathcal{L}_{S_\tau^{\text{s}}}(f_{\mathbf{I},\boldsymbol{\eta}})$. One numerical fitting rule uses the damped direction and finite line search

$$\begin{aligned} \mathbf{g} &:= \nabla F_{\tau,\mathbf{I}}(\boldsymbol{\eta}), \qquad \boldsymbol{\Delta} := -(\mathbf{H} + \lambda \mathbf{Id})^{-1}\mathbf{g}, \\ \mathcal{S}(\boldsymbol{\eta}, \boldsymbol{\Delta}) &:= \left\{\beta^l : 0 \leq l < N_{\text{ls}},\ F_{\tau,\mathbf{I}}(\boldsymbol{\eta} + \beta^l \boldsymbol{\Delta}) < F_{\tau,\mathbf{I}}(\boldsymbol{\eta})\right\}, \\ \alpha &:= \max\big(\{0\} \cup \mathcal{S}(\boldsymbol{\eta}, \boldsymbol{\Delta})\big), \end{aligned} \tag{13}$$

where $\mathbf{H} = \mathbf{H}_{\tau,\mathbf{I}}(\boldsymbol{\eta}) \succeq 0$, $\lambda > 0$, $0 < \beta < 1$, and N_{ls} bounds the line search. For mean squared error, let $\mathbf{r}$ stack the n_τ scalar prediction residuals on support data. The Gauss–Newton choice is

$$\mathbf{J} := \frac{\partial \mathbf{r}}{\partial \boldsymbol{\eta}}, \qquad \mathbf{H}_{\tau,\mathbf{I}}(\boldsymbol{\eta}) := \frac{2}{n_\tau}\mathbf{J}^{\mathsf{T}}\mathbf{J}. \tag{14}$$

The choice $\mathbf{H} = \mathbf{0}$ gives a scaled gradient step. Only independent trainable coordinates are included in $\boldsymbol{\eta}$.

For each candidate and task, several initial states $\{\boldsymbol{\eta}_{\tau,r}^{(0)}\}_{r=1}^{R}$ generate a finite set of iterates. They can include that task's source-derived initializations. Numerical states and the shared structure are selected by

$$\begin{aligned} (\widehat{r}_\tau, \widehat{k}_\tau)(\mathbf{I}) &\in \arg\min_{r,k} \mathcal{L}_{V_\tau^{\text{num}}}(f_{\mathbf{I},\boldsymbol{\eta}_{\tau,r}^{(k)}}), \\ \widehat{\mathbf{I}} &\in \arg\min_{\mathbf{I}\in\mathcal{B}} \frac{1}{T}\sum_{\tau} \mathcal{L}_{V_\tau^{\text{str}}}(f_{\mathbf{I},\widehat{\boldsymbol{\eta}}_\tau(\mathbf{I})}). \end{aligned} \tag{15}$$

Here $\widehat{\boldsymbol{\eta}}_\tau(\mathbf{I})$ is the iterate selected by the first line. The bank and these iterates are fixed before the second line is evaluated.

On a target task, $\widehat{\mathbf{I}}$ is fixed and all its numerical parameters are initialized afresh and fitted on the target support sample. Query loss measures transfer. Recovery depends on whether the finite bank contains a useful structure and whether numerical fitting can reveal it.

4.4 ReLU implementation

Our experimental network is

$$f_\tau(\mathbf{x}) = H^{-1/2}\mathbf{A}_\tau^{\mathsf{T}}\rho(\mathbf{W}_\tau^{\mathsf{T}}\mathbf{x} + \mathbf{b}_\tau) + \mathbf{d}_\tau, \tag{16}$$

where $\mathbf{W}_\tau \in \mathbb{R}^{D\times H}$, $\mathbf{A}_\tau \in \mathbb{R}^{H\times O}$, $\mathbf{b}_\tau \in \mathbb{R}^H$, $\mathbf{d}_\tau \in \mathbb{R}^O$, and ρ is coordinatewise ReLU. Only $\mathbf{W}_\tau$ is structured; the biases and output layer are free.

For synthetic regression, dense solutions are refined by Gauss–Newton steps and residual-based reinitialization of weak neurons. Normalized columns $\mathbf{r}_{\tau,h} := \widetilde{\mathbf{W}}_{\tau,:,h}/\|\widetilde{\mathbf{W}}_{\tau,:,h}\|_2$ define

$$z_{\tau,dh}(\mathbf{s}_\tau) = s_{\tau,h} r_{\tau,dh}, \qquad u_{\tau,dh}(\mathbf{s}_\tau) = \max\{|s_{\tau,h}|, \varepsilon_s\}\max\{\widehat{u}_{\tau,h}, \varepsilon_u\}, \tag{17}$$

Algorithm 2: Numerical adaptation and structure selection

Input: Bank $\mathcal{B}$; source samples; numerical starts; fitting budget N_{fit}.
Output: Shared indexator $\widehat{\mathbf{I}}$.

foreach $\mathbf{I} \in \mathcal{B}$ *and* $\tau \in \mathcal{T}_{\mathrm{tr}}$ **do**
  $b_\tau(\mathbf{I}) \leftarrow +\infty$
  for $r = 1, \dots, R$ **do**
    $\boldsymbol{\eta} \leftarrow \boldsymbol{\eta}^{(0)}_{\tau,r}$
    for $k = 0, \dots, N_{\mathrm{fit}}$ **do**
      $v \leftarrow \mathcal{L}_{V^{\mathrm{num}}_\tau}(f_{\mathbf{I},\boldsymbol{\eta}})$
      if $v < b_\tau(\mathbf{I})$ **then**
        $(b_\tau(\mathbf{I}), \widehat{\boldsymbol{\eta}}_\tau(\mathbf{I})) \leftarrow (v, \boldsymbol{\eta})$
      if $k = N_{\mathrm{fit}}$ **then**
        break
      $\mathbf{g} \leftarrow \nabla F_{\tau,\mathbf{I}}(\boldsymbol{\eta})$
      $\mathbf{H} \leftarrow \mathbf{H}_{\tau,\mathbf{I}}(\boldsymbol{\eta})$
      $\boldsymbol{\Delta} \leftarrow -(\mathbf{H} + \lambda \mathbf{Id})^{-1}\mathbf{g}$
      $\alpha \leftarrow \max(\{0\} \cup \mathcal{S}(\boldsymbol{\eta}, \boldsymbol{\Delta}))$
      if $\alpha = 0$ **then**
        break
      $\boldsymbol{\eta} \leftarrow \boldsymbol{\eta} + \alpha\boldsymbol{\Delta}$
// Structure-selection data enter only after all numerical choices.
$s(\mathbf{I}) \leftarrow \dfrac{1}{T}\sum_\tau \mathcal{L}_{V^{\mathrm{str}}_\tau}(f_{\mathbf{I},\widehat{\boldsymbol{\eta}}_\tau(\mathbf{I})})$ for each $\mathbf{I} \in \mathcal{B}$
$\widehat{\mathbf{I}} \in \arg\min_{\mathbf{I}\in\mathcal{B}} s(\mathbf{I})$
return $\widehat{\mathbf{I}}$

with positive floors $\varepsilon_s, \varepsilon_u$. Reliability weights $\widehat{u}_{\tau,h}$ are estimated from residuals and the pseudoinverse of the empirical Jacobian Gram matrix, propagated through column normalization.

For anchor directions $\bar{\mathbf{r}}_{\tau,h}$ and support $\mathbf{M}^{\mathrm{a}}$, hidden-unit alignment solves

$$\begin{aligned} \pi_\tau &\in \arg\min_{\pi\in\mathfrak{S}_H} \sum_{h=1}^{H} D_{\tau,h,\pi(h)}, \\ D_{\tau,hk} &:= \sum_d \mathbf{1}\{M^{\mathrm{a}}_{dh} \neq \widetilde{M}_{\tau,dk}\} + \omega \min_{\sigma\in\{-1,1\}} \|\bar{\mathbf{r}}_{\tau,h} - \sigma \mathbf{r}_{\tau,k}\|_2^2, \end{aligned} \tag{18}$$

where $\mathfrak{S}_H$ is the permutation set, $\widetilde{\mathbf{M}}_\tau$ is the thresholded source support, and $\omega > 0$ weights direction agreement. Calibration fixes one column scale per connected component of the code–neuron incidence graph, connecting each code to the neurons using it.

The implemented signed codebook has m independent values:

$$\mathbf{C}_\tau = (0, \mathbf{c}_\tau^{\mathsf{T}}, -\mathbf{c}_\tau^{\mathsf{T}})^{\mathsf{T}}, \qquad K = 2m. \tag{19}$$

For this restricted numerical family, replace $\mathbf{A}_{\mathbf{I}}$ by the signed lookup matrix with row $\sigma\mathbf{e}_j^{\mathsf{T}}$ for code $\sigma c_{\tau,j}$, where $\mathbf{e}_j$ is the jth standard basis vector. Writing its signed labels as $J_{dh} \in \{0, \pm1, \dots, \pm m\}$, each reference source t yields the oriented proposal $J^{(t)}_{dh} = \sigma_{t,h} J_{dh}$, where $\sigma_{t,h} := 2\mathbf{1}\{s_{t,h} \geq 0\} - 1$. Reversing a ReLU active side is checked by functional fitting. Canonicalization permutes and changes signs of code pairs jointly. Empty independent groups are reseeded from groups containing several positions.

For support inputs $\{\mathbf{x}_n\}$, define $\Phi_{nh} := H^{-1/2}\rho(\mathbf{x}_n^{\mathsf{T}}\mathbf{W}_{\tau,:,h} + b_{\tau,h})$ and let $\mathbf{Y}$ collect their targets. For squared-loss refinement, the free output layer is refitted at every iterate and line-search trial by

$$\begin{pmatrix} \mathbf{A}_\tau \\ \mathbf{d}_\tau^{\mathsf{T}} \end{pmatrix} \in \arg\min_{\mathbf{U}} \left\{ \|[\boldsymbol{\Phi}, \mathbf{1}]\mathbf{U} - \mathbf{Y}\|_{\mathrm{F}}^2 + \varepsilon_{\mathrm{lin}} \|\mathbf{U}\|_{\mathrm{F}}^2 \right\}, \tag{20}$$

with a small stabilizer $\varepsilon_{\mathrm{lin}} > 0$. Hidden biases and codes remain trainable. Gauge normalization uses $a = \|\mathbf{c}_\tau\|_2/\sqrt{m}$ and $(\mathbf{c}_\tau, \mathbf{b}_\tau, \mathbf{A}_\tau) \leftarrow (\mathbf{c}_\tau/a, \mathbf{b}_\tau/a, a\mathbf{A}_\tau)$ for $a > 0$. These descriptor, alignment and solver

choices specify the ReLU implementation of the general projection-and-selection procedure. For the real-data experiments we use Adam numerical fitting and direction dispersion across independently fitted models as discrepancy scales. These changes preserve the shared projection and functional selection stages; their finite search budgets are reported separately.

5 Experiments

We evaluate transfer of structural constraints. A target task receives the selected indexator, or a competitor's native continuous basis; its codebook, hidden biases, output matrix and output offset are learned from fresh initialization. Source numerical values are never transferred in the primary comparison. All methods use the same fully connected ReLU architecture, with structure restricted to the first weight matrix.

5.1 Data and selection

Table 1 lists the four benchmarks. The paired dictionary has m independent values and $2m + 1$ lookup symbols, including zero. The prescribed m is available on synthetic data; no true weights, biases, mask, sharing groups, coordinate roles or noise levels enter learning. Real data have no reference neural-network indexator. Source structure-selection loss chooses a candidate within each fixed budget; separate development tasks select m from the same registered grid for every method.

Benchmark	D	H	m	Source tasks	Target tasks
Pattern 0110	11	8	2	3×16	3×8
Synthetic DeepSets	16	16	4	3×16	3×8
Aug-Omniglot	784	16	$8, 16, 32, 64$	128	1000
Paired MNIST sums	98	32	$16, 32, 64, 128$	16	32

Tab. 1: Network dimensions, independent code budgets and task counts. Synthetic counts cover three independent data/numerical draws; the same target tasks evaluate structures learned with clean and noisy source observations.

5.1.0.1 Synthetic tasks.

Pattern tasks use a sliding 0110 detector and independent numerical values. Synthetic DeepSets tasks sum repeated local two-feature ReLU maps, with an unknown permutation of input coordinates. Hidden-unit orders are randomized. Each source supplies 4096 fitting examples, 512 numerical-selection examples and 1024 separate structure-selection examples. Input scales $0.5, 1, 2, 4$ are mixed during source fitting; evaluation inputs are standard Gaussian. Source noise standard deviations are 0 and 0.1. Target functions have independently drawn numerical values and clean observations. Their support sample is split into $N \in \{64, 256, 1024\}$ fitting and $N/4$ numerical-selection examples; 2048 further examples measure prediction error. A separate source pilot precedes the reported confirmation series.

5.1.0.2 Aug-Omniglot.

We use inverted 28×28 Omniglot images [17], the standard 1028/172/423 base-character split and four virtual right-angle rotations. Episodes are 5-way 5-shot with 15 queries per class. Query-only random crops, horizontal/vertical flips and rotations follow the augmentation setting of MSR [6]. The fixed source cohort contains 128 episodes; source queries are divided into five numerical-selection and ten structure-selection examples per class. Another 128 episodes from development classes choose the code budget and a global

numerical learning rate, initialization rule and update count, averaging two fresh restarts. Each test episode then fits only its 25 support observations.

5.1.0.3 Paired MNIST sums.

This reduced multitask extension of the image-sum problem in Deep Sets [13] uses two MNIST images per example, average-pooled to 7×7, flattened and subjected to one hidden coordinate permutation. A task assigns a fresh scalar value to each digit and predicts the sum of the two values. Source, development and target-support image pools partition the official training set into 40k/10k/10k images; target queries use the official test set. Source fitting/numerical-selection/structure-selection sizes are 4096/512/1024. Thirty-two development tasks choose the code budget and global numerical rule for a 512-example target support sample. This is a reduced, fixed-set-size extension, rather than the published variable-size InfiMNIST experiment; published scores are not matched comparisons.

5.2 Competitors and numerical controls

The categorical $A\varphi$ baseline follows Yeh et al. [5]: a shared row-softmax assignment matrix and task-specific numerical parameters are optimized through a support/numerical-selection implicit hypergradient. We report the native continuous basis and its MAP indexator separately. MSR-FC [6] uses an unrestricted shared $U\varphi$ parameterization and full higher-order MAML, with learned numerical initialization and inner step sizes. Its primary transfer result keeps only U. LUT-Q [15] uses nearest-code forward weights, straight-through latent updates and one Lloyd update per minibatch. Its source indexators are retained directly. Competitors receive no additional clustering step. The signed dictionary and common fully connected task family are explicit adaptations of the original methods. An auxiliary native MSR control also transfers its learned numerical initialization and SGD rates and takes three inner steps. It is reported separately because it uses more source knowledge than structure alone.

Every discrete bank and continuous basis receives a common numerical refitting budget before independent source structure selection. Fully connected Adam is included throughout. Synthetic evaluation also includes fully connected Adam followed by the same Gauss–Newton refinement used for structured models. Random-indexator controls shuffle the selected indexator's entries, retaining its observed connection and group counts without using the true mask. Learning rates, restarts and numerical checkpoints are selected using the designated development data; target query responses remain closed until predictors and predictions are frozen.

The synthetic extractor is the unchanged G34 implementation. For images, code elimination reduces its weighted calibration system to a column-scale Schur system, preserving the projection objective. The finite real-data bank uses two dense anchors, direction dispersion among four fitted starts, four support proposals and 40 assignment iterations; only exact duplicate arrays are removed at large m. Real numerical fitting uses Adam. These implementation choices and their full search costs are recorded with the results.

5.3 Measurements

Parameter counts include every numerical coordinate fitted on one target task. For a structured first layer this is $m + H + HO + O$, and for Dense it is $DH + H + HO + O$, including hidden biases, the free output matrix and output offset. The fixed transferred indexator or basis contributes no target trainable parameters. These are model coordinates, not source-search parameters or the total across restarts. Mask controls count active weights; their optimizer's larger allocated tensor is recorded separately.

Prediction error is measured on new tasks. Synthetic recovery is audited only after freezing predictions and is reported modulo code-pair renaming/sign changes and hidden-unit permutations. This is a specified symmetry test, not a decision procedure for all functional equivalences. Continuous competitors are not

assigned a discrete recovery score. Uncertainty across synthetic cases is computed at the family level; tasks sharing an extracted structure are not treated as independent structure discoveries.

Offline extraction/search and fresh target adaptation are measured separately on V100 32GB GPUs. We report synchronized stage wall time, per-process peak CUDA allocation and registered PyTorch-operator FLOPs. The operation count excludes NumPy/SciPy work and unsupported operators and therefore is incomplete. Target seconds per task are amortized throughput under packed execution, rather than single-task latency. Search includes all tried configurations and required dense preparation; unstructured target fitting requires no source extractor. Target measurements include adaptation and prediction on query inputs; peak allocation covers the whole packed process. They are not model storage sizes. The synthetic Adam-only clock stops before Gauss–Newton.

5.4 Results

Method	**Pattern** (0)	**Pattern** (0.1)	**DeepSets** (0)	**DeepSets** (0.1)
Dense Adam	1.36e-02	1.36e-02	1.62e-02	1.62e-02
Dense Adam + GN	6.90e-03	6.90e-03	1.52e-02	1.52e-02
Random I	8.05e-02	8.05e-02	1.54e-01	1.42e-01
LUT-Q I	$< 10^{-12}$	$< 10^{-12}$	1.29e-04	3.86e-03
Yeh MAP I	7.83e-04	6.79e-03	2.73e-02	3.05e-02
Yeh soft	3.28e-04	4.76e-04	1.92e-02	2.91e-02
MSR-FC U	3.05e-03	8.11e-03	5.83e-02	6.15e-02
Ours	$< 10^{-12}$	$< 10^{-12}$	6.10e-04	5.36e-03

Tab. 2: Mean target MSE over three families, each with eight new numerical tasks. Parentheses in headings give source noise; target data are clean. All models use 1024 fitting and 256 numerical-selection observations. Family-level spreads and all smaller support budgets are in the accompanying CSV.

Method	**Omniglot accuracy (%)**	m	**MNIST MSE**	m
Dense Adam	37.14 ± 0.49	–	1.058	–
Random I	25.01 ± 0.36	64	1.234	128
LUT-Q I	26.15 ± 0.43	8	1.231	128
Yeh MAP I	34.23 ± 0.47	64	1.227	128
Yeh soft	32.45 ± 0.49	64	1.201	128
MSR-FC U	34.62 ± 0.48	64	1.193	128
Ours	37.73 ± 0.56	64	1.185	128
Ours + functional	–	–	1.192	128
Our mask, free weights	–	–	1.062	627
Random mask, free weights	–	–	1.145	627
MSR + learned init	31.58 ± 0.48	8	1.159	32

Tab. 3: Real-data transfer with fresh task parameters. Omniglot shows 95% t intervals across episodes, conditional on one source cohort. MNIST uses 32 new digit-value tasks. Mask-only controls use more independent values, explicitly shown by m. The final row additionally transfers learned numerical initialization and is a separate regime. Conditional ablations are tested only on MNIST.

Method	Pattern	DeepSets	Omniglot	MNIST
Dense	105	306	12645	3201
Random I	19	54	165	193
LUT-Q I	19	54	109	193
Yeh MAP I	19	54	165	193
Yeh soft	19	54	165	193
MSR-FC U	19	54	165	193
Ours	19	54	165	193
Ours + functional	–	–	–	193
Our mask, free weights	–	–	–	692
Random mask, free weights	–	–	–	692
MSR + learned init	–	–	109	97

Tab. 4: Total numerical parameters fitted per target task, including hidden biases, the free output matrix and output offset. Dense Adam and Dense Adam plus GN have identical parameter counts. The fixed transferred structure is excluded. Mask controls count active weights; their current optimizer still allocates the full masked first-layer tensor (3201 coordinates on MNIST), distinguished in CSV. The native MSR row transfers learned initialization.

Method	Pattern 0	Pattern 0.1	DeepSets 0	DeepSets 0.1
Ours	3/3	3/3	3/3	0/3
LUT-Q I	3/3	3/3	2/3	0/3
Yeh MAP I	0/3	1/3	0/3	0/3
Random I	0/3	0/3	0/3	0/3
Our bank coverage	3/3	3/3	3/3	0/3

Tab. 5: Exact generating indexator recovery under the stated code-pair/sign/hidden-unit symmetries, three cases per cell. Bank coverage is a late descriptive audit, not an oracle selection rule. Full mask precision/recall and sharing ARI are supplied in CSV. Continuous bases have no discrete recovery score.

5.4.0.1 Recovery and prediction are different criteria.

Our method recovers the generating Pattern indexator in all six cases and the clean DeepSets indexator in all three cases (Table 5). LUT-Q matches the Pattern result and recovers two of the three clean DeepSets indexators. Nevertheless, LUT-Q has lower clean DeepSets query MSE, 1.29×10^{-4} versus 6.10×10^{-4}. Finite numerical fitting can leave residual error even when the selected structure admits the generating function; recovery alone does not guarantee that optimization reaches it. Figure 1 separates sample size from numerical training progress.

With source noise 0.1, neither method exactly recovers any of the three DeepSets indexators. Our frozen banks contain no generating indexator in these cases, so functional selection cannot recover it from those banks. Two selected masks remain exact; the third contains 117 active connections instead of 32. Thus the remaining failure includes candidate coverage and sharing, rather than only codebook optimization. Figure 2 displays the first registered family under both noise conditions. Binary Pattern follow-up results appear in Appendix A.3.

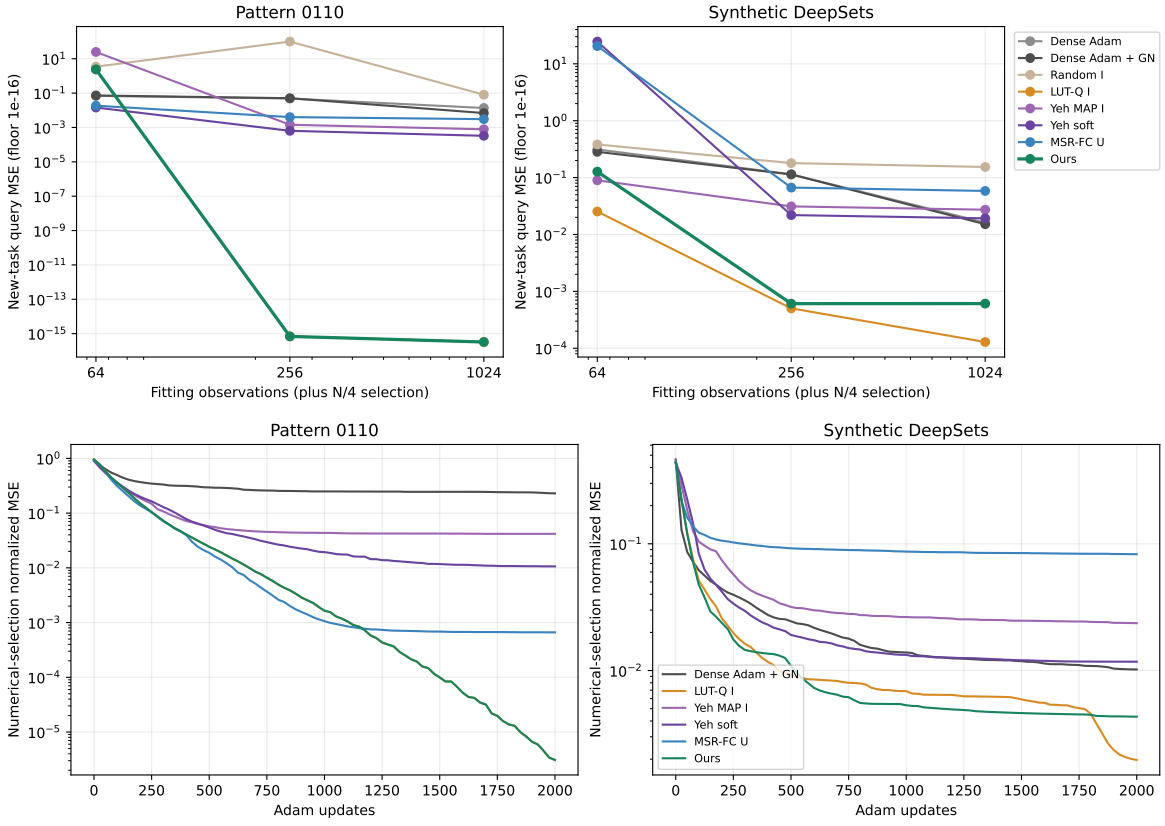


Fig. 1: Top: clean-source transfer MSE over three families at different support sizes. Bottom: numerical-selection loss during Adam for the first registered family, taking the best checkpoint and restart so far and averaging eight new tasks. The lower panels precede Gauss–Newton and are not test-loss curves.

5.4.0.2 Real-data transfer.

Our Omniglot accuracy is 37.73% with 64 independent first-layer values, compared with Dense at 37.14% with 12544 first-layer weights. The selected mask is fully connected; the transferred restriction here is sharing rather than sparsity. The paired difference is +0.59 percentage points, with a 95% interval of approximately $[-0.02, 1.19]$, conditional on one source cohort. This supports competitive transfer with a smaller numerical family, but does not establish an accuracy advantage over Dense. All absolute accuracies

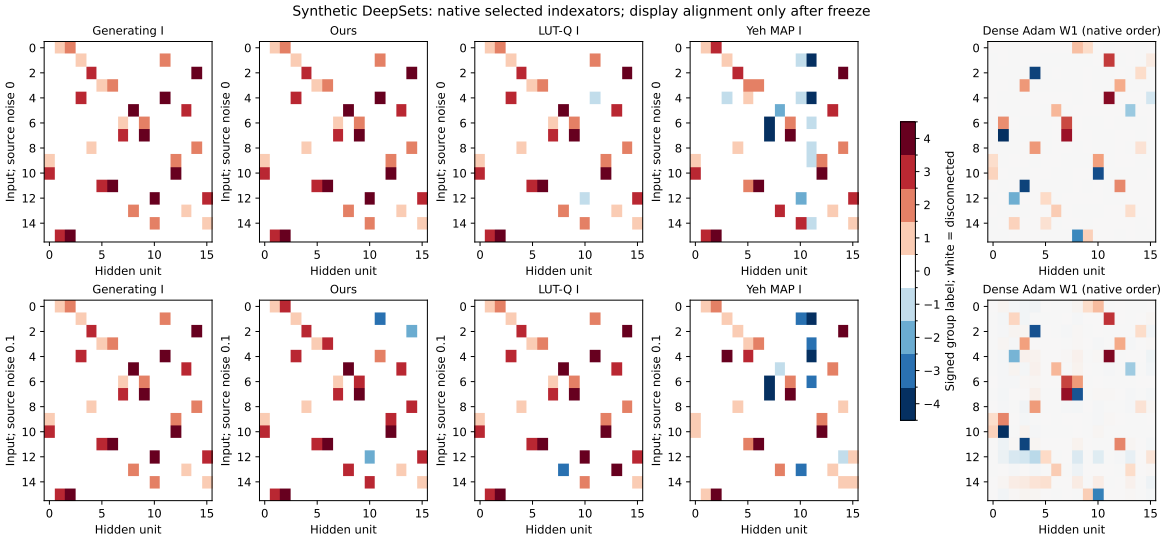


Fig. 2: First registered DeepSets family. Discrete labels are aligned with the generating indexator only for this late audit; zero is white. Equal signed labels denote the same signed code. The right column shows unconstrained Adam weights in their native hidden-unit order, with an independent numerical scale.

concern the narrow fully connected setting here. The published MSR result of $97.7 \pm 0.2\%$ on augmented 5-way 5-shot Omniglot [6] uses a CNN, learned numerical initialization and a much larger meta-training budget; it is contextual evidence, not a matched row in Table 3.

On paired MNIST sums, our 128-code structure has MSE 1.185 versus Dense at 1.058, a paired increase of 0.127 with a 95% interval of approximately $[0.087, 0.167]$. The other tested 128-dimensional structural families also trail Dense. This experiment does not demonstrate useful sharing at the tested code budgets.

5.4.0.3 Conditional diagnosis on MNIST.

Before opening target responses, development loss revealed an 11.1% gap to Dense. We therefore evaluated the source-only hard-assignment refinement in Appendix A.1. Its own source numerical-selection MSE fell from 0.574 to 0.433, but development transfer worsened from 1.203 to 1.238. It was not adopted. Its frozen test MSE, 1.192, also fails to improve the original 1.185.

Keeping the learned mask and releasing equality constraints gives test MSE 1.062, close to Dense at 1.058. A random mask preserving every hidden neuron's degree gives 1.145. This isolates useful connectivity from the limitations of the selected sharing constraints at 128 codes. The mask control has 627 independent first-layer values, versus 128 for sharing and 3136 for Dense, so it is a larger-budget diagnostic. It does not verify a true neural indexator, which is unavailable on real data. Figure 3 includes all conditional controls rather than replacing the primary row with a favorable one.

5.4.0.4 Computation.

Full measurements are in Appendix A.4. On the representative DeepSets family, fresh adaptation plus prediction takes 0.993 seconds per task for our structure versus 1.212 for Dense Adam plus Gauss–Newton; Dense Adam alone takes 0.775 seconds and has higher error. On Omniglot, our packed target cost is 2.75 ms per task versus Dense at 1.58 ms. On MNIST it is 29.7 versus 30.9 ms, but peak target allocation is 1096 versus 71 MiB: the current basis implementation materializes large intermediate feature tensors. Fewer independent parameters therefore do not imply universally faster or smaller-memory execution. Extraction is an additional offline cost, and the incomplete operator counts must not be interpreted as total hardware FLOPs.

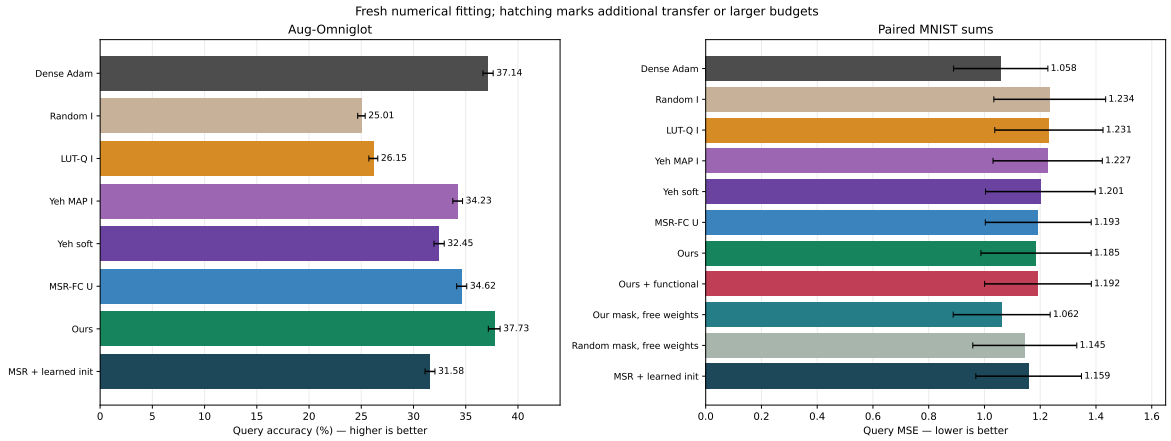


Fig. 3: Real-data transfer. Intervals describe episode/task variability conditional on one source cohort. Hatching marks larger numerical budgets or transfer of learned initialization; these are separate controls.

5.4.0.5 Scope.

Three synthetic families and one source cohort per real benchmark do not quantify variability over arbitrary task environments. The real protocols, signed dictionaries and common fully connected architectures are disclosed adaptations of the original work. Our results support exact extraction and transfer in the clean synthetic setting and competitive few-shot transfer on the reduced Omniglot setting, while exposing failures under noise and restrictive real-data sharing.

6 Conclusion

We extract a common connectivity and weight-sharing constraint by aligning source solutions, jointly projecting their task-specific weights and selecting a finite bank by prediction error after numerical refitting. New tasks receive only the selected constraint; their numerical parameters are learned afresh.

The method recovers the generating structure on the tested Pattern and clean DeepSets families. Recovery degrades under source noise, and lower-dimensional sharing does not consistently improve real-data prediction or execution cost. On paired MNIST, retaining connectivity while releasing equality constraints nearly matches Dense, whereas functional refinement of the hard assignments does not improve transfer. These results make candidate coverage, noise tolerance and the choice of sharing strength the main unresolved limitations.

A Additional benchmark details

A.1 Functional hard-assignment control

This conditional MNIST control starts from the source-selected indexator and its fitted source parameters. Let J be its signed representation, and let V_τ^{num} be the source numerical-selection sample. At fixed task parameters, write

$$g_{\tau,p} := \frac{\partial \mathcal{L}_{V_\tau^{\mathrm{num}}}}{\partial w_{\tau,p}}, \qquad h_{\tau,p} := \frac{2}{n_\tau O} \sum_{n,o} \left(\frac{\partial f_{\tau,o}(x_n)}{\partial w_{\tau,p}} \right)^2, \tag{21}$$

where $n_\tau = |V_\tau^{\mathrm{num}}|$ and O is the output dimension. For a replacement $j \in \{0, \pm 1, \dots, \pm m\}$, define $q_\tau(j) := \mathrm{sign}(j) c_{\tau,|j|}$, with $q_\tau(0) = 0$, and $\delta_{\tau,pj} := q_\tau(j) - q_\tau(J_p)$. The shared proposal is

$$j_p^+ \in \arg\min_j \frac{1}{T} \sum_\tau \left(g_{\tau,p} \delta_{\tau,pj} + \frac{1}{2} h_{\tau,p} \delta_{\tau,pj}^2 \right). \tag{22}$$

This diagonal Gauss–Newton score is a proposal surrogate, not an exact loss change for a nonlinear network or simultaneous replacements. The actual numerical-selection loss must decrease before a proposed packet is accepted. Write $R(J, \eta) := T^{-1} \sum_\tau \mathcal{L}_{V_\tau^{\mathrm{num}}}(f_{J,\eta_\tau})$.

Algorithm 3: Conditional functional proposal refinement

Input: Source-selected J, fitted numerical parameters $\{\eta_\tau\}$, source fitting and numerical-selection data.

$\mathcal{B}_{\mathrm{f}} \leftarrow \{J\}$
for $k = 1, \dots, 400$ **do**
  $\eta_\tau \leftarrow 20$ Adam updates on $\mathcal{L}_{S_\tau^{\mathrm{s}}}$ at fixed J, for all τ
  Compute $g_{\tau,p}, h_{\tau,p}$ and j_p^+ by (21)–(22)
  Order profitable replacements by their predicted loss decrease
  $a \leftarrow 0$
  for $b \in (64, 32, 16, 8, 4, 2, 1)$ **do**
    $J' \leftarrow J$ with its first b profitable replacements
    if $R(J', \eta) < R(J, \eta)$ **then**
      $(J, a) \leftarrow (J', 1)$; **break**
  if $a = 0$ **then**
    $\mathcal{N}_{16} \leftarrow \{J$ with one of its first 16 profitable replacements$\}$
    `// Actual loss rescues a misleading quadratic ordering.`
    $J \in \arg\min_{J' \in \mathcal{N}_{16} \cup \{J\}} R(J', \eta)$
  Retain distinct trajectory checkpoints in $\mathcal{B}_{\mathrm{f}}$
Refit the finite bank and select by (15)

Four numerical learning rates, $0.0003, 0.001, 0.003, 0.01$, are run; seven distinct candidates including the unchanged baseline receive common refitting. Because the starting indexator was selected using source structure-selection data, its reuse here makes those source losses exploratory; the disjoint development and final target cohorts remain separate. A preliminary attempt with an incorrect basis-scale conversion was excluded before candidate export. In the corrected attempt, the initial source predictor and its loss agree in both representations. No target responses or reference parameters enter this procedure. Development results rejected it before test scoring.

A.2 Numerical and competitor controls

Synthetic target adaptation uses 18 fresh Adam starts: learning rates $0.003, 0.01, 0.03$, three bias-activity quantiles $0.1, 0.5, 0.9$, and two random starts per setting. Numerical-selection data choose checkpoints over

2000 updates; four retained starts receive up to 48 Gauss–Newton updates. Dense receives the same budget, with its Adam checkpoint reported separately. On real data, development tasks select one global learning rate, quantile and update count; a target task uses a single fresh start and only its support responses. The selected update counts are 25/50 for Dense/ours on Omniglot and 225/250 on MNIST.

A stronger Yeh source-only control on one separate DeepSets pilot uses 100 rather than 20 following inner steps and an explicit spectrally damped inner Hessian solve rather than a 20-term Neumann action. Its source structure-selection MSE improves from 0.04854 to 0.04325 for MAP and from 0.02905 to 0.02507 for the continuous basis; our pilot value is 0.001762. This limited control does not establish optimal tuning of Yeh, and these source losses are not target results.

The image implementation eliminates code variables from the weighted calibration solve before solving for column scales. Comparison against the unreduced system on learned source descriptors gives relative code difference 3.50×10^{-13}. Thus this computational reduction preserves that conditional calibration objective; it does not add structural information.

A.3 Binary Pattern and additional heatmaps

Method	Clean-source I	Noisy-source I
Dense Adam	91.17	91.17
Random I	58.43	58.43
LUT-Q I	100.00	100.00
Yeh MAP I	100.00	96.59
Yeh soft	100.00	100.00
MSR-FC U	99.31	95.83
Ours	100.00	100.00

Tab. 6: Binary Pattern follow-up: mean query accuracy (%) over three families, with 512 support, 128 numerical-selection and 1408 held-out binary inputs. All numerical parameters are trained afresh by binary cross entropy. These structures were extracted from real-valued source functions, so this is not extraction from binary supervision alone.

A.4 Measured computational costs

Offline costs include required dense preparation, all configurations, common source refitting and development selection. Continuous competitors pay only for the dense preparation they use, rather than the extra descriptor refinement required by discrete refitting. The conditional functional row additionally includes the original search and its diagnostic/refinement stages. Shared stages are counted in each independent pipeline; summing rows would count them repeatedly. Search seconds are measured with operator profiling under shared scheduling and are not an isolated latency benchmark. The CPU parts of calibration are timed but absent from the registered Torch counts.

Target timings are separate unprofiled runs, one process per GPU. Synthetic rows measure the first registered family at 1024 fitting examples; real rows measure the complete target cohort. Allocated memory depends on packing: eight tasks for synthetic data and chunks of up to 64 for images. Synthetic target operator counts in the CSV cover all three support sizes, whereas its timing column covers only 1024; these scopes must not be combined as a single FLOP rate. An unavailable Adam-only peak is left blank rather than replaced by the later Gauss–Newton peak. The target GFLOPs column reports work per task, including all numerical starts and updates in the stated scope. FLOP/s is a throughput measure and is not measured here. Unsupported Torch operations and CPU numerical linear algebra are excluded from these counts.

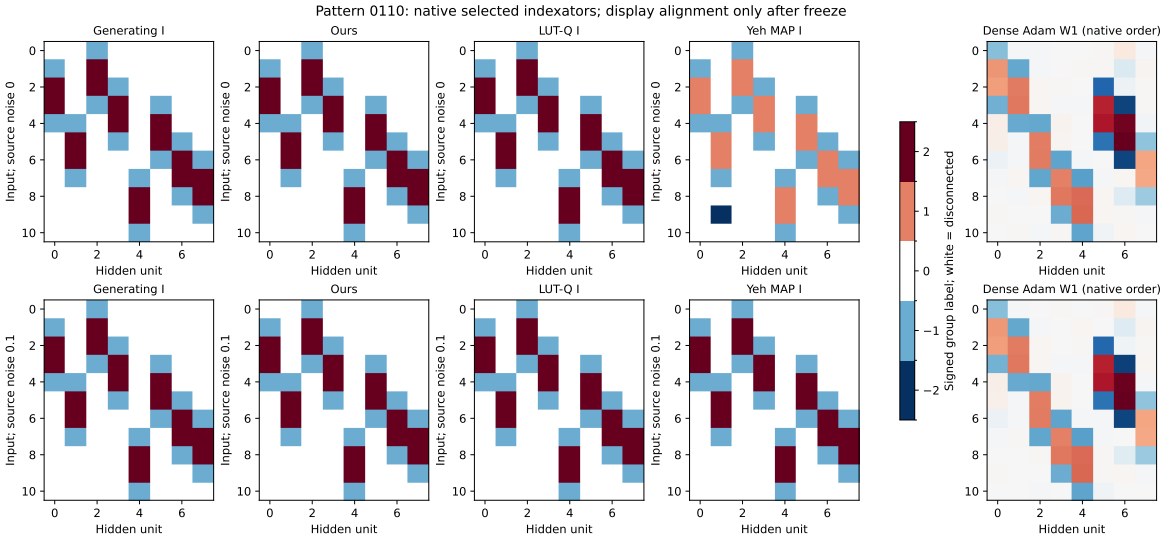


Fig. 4: Pattern indexators for the first registered family. Display alignment uses reference labels only after all predictions are frozen. Dense weights retain native unit order.

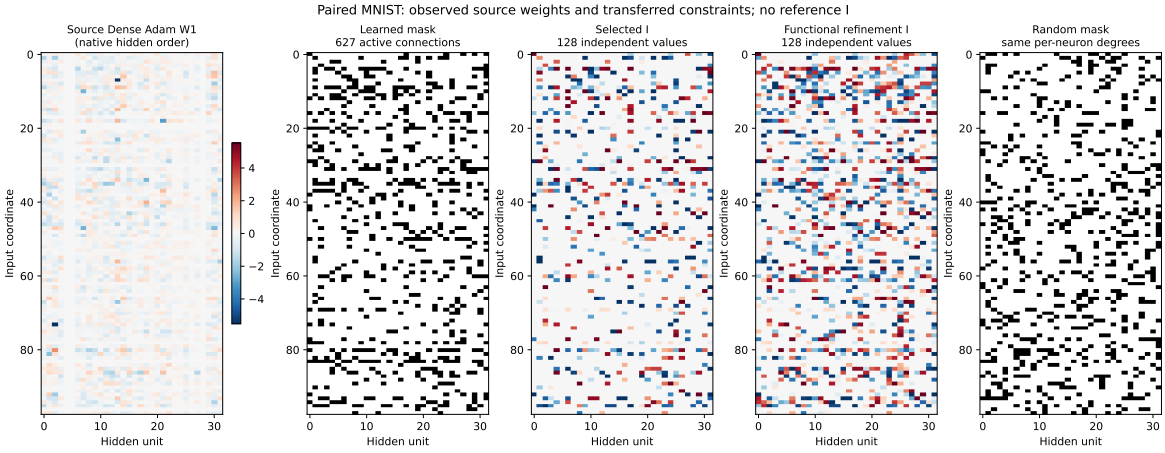


Fig. 5: Paired MNIST source weights, learned mask, selected signed indexator, functional control and degree-matched random mask. Group labels are names, not numerical weight magnitudes. Releasing weights in the learned mask changes the numerical budget from 128 to 627; no reference indexator is available.

Method	Search s*	Search TFLOPs*	Target GFLOPs†	ms/task	Target MiB	Search MiB
Dense Adam	0	0	–	788	–	0
Dense Adam + GN	0	0	28.2	996	650	0
Random I	–	–	4.5	969	44.5	–
LUT-Q I	198	2.86	4.52	948	44.5	926
Yeh MAP I	443	3.43	4.52	955	44.5	926
Yeh soft	364	1.43	4.49	955	44.5	833
MSR-FC U	183	0.956	4.51	901	44.5	833
Ours	119	1.97	4.52	963	44.5	926

Tab. 7: Pattern 0110: measured computation. Target timing excludes the FLOP profiler and uses one process per GPU. *Search seconds include profiling overhead and shared scheduling. †Target GFLOPs are per task: the complete 64/256/1024 sweep on synthetic data, one prescribed support budget on real data; synthetic time covers only 1024. These are incomplete Torch operation counts, not FLOP/s. A dash indicates unavailable measurement.

Method	Search s*	Search TFLOPs*	Target GFLOPs†	ms/task	Target MiB	Search MiB
Dense Adam	0	0	–	775	–	0
Dense Adam + GN	0	0	189	1212	3668	0
Random I	–	–	19.8	948	136	–
LUT-Q I	293	20.5	20.1	943	136	3877
Yeh MAP I	556	21.9	19.8	959	136	3877
Yeh soft	400	6.09	19.9	966	136	3611
MSR-FC U	191	4.5	19.9	962	136	3611
Ours	533	20.4	20.2	993	136	3877

Tab. 8: Synthetic DeepSets: measured computation. Target timing excludes the FLOP profiler and uses one process per GPU. *Search seconds include profiling overhead and shared scheduling. †Target GFLOPs are per task: the complete 64/256/1024 sweep on synthetic data, one prescribed support budget on real data; synthetic time covers only 1024. These are incomplete Torch operation counts, not FLOP/s. A dash indicates unavailable measurement.

Method	Search s*	Search TFLOPs*	Target GFLOPs†	ms/task	Target MiB	Search MiB
Dense Adam	31.9	6.31	0.0354	1.58	70.5	1067
Random I	–	–	0.164	1.66	59.1	–
LUT-Q I	1704	20.9	0.0214	2.56	56.8	2426
Yeh MAP I	3025	103	0.167	2.78	59.1	1176
Yeh soft	3084	104	0.164	1.52	59.1	1176
MSR-FC U	2080	55.6	0.167	2.59	59.1	705
Ours	1313	15.9	0.167	2.75	59.1	705
MSR + learned init	554	40.4	7.10e-03	0.476	57.5	705

Tab. 9: Aug-Omniglot: measured computation. Target timing excludes the FLOP profiler and uses one process per GPU. *Search seconds include profiling overhead and shared scheduling. †Target GFLOPs are per task: the complete 64/256/1024 sweep on synthetic data, one prescribed support budget on real data; synthetic time covers only 1024. These are incomplete Torch operation counts, not FLOP/s. A dash indicates unavailable measurement.

Method	Search s*	Search TFLOPs*	Target GFLOPs†	ms/task	Target MiB	Search MiB
Dense Adam	27.8	7.74	1.52	30.9	71.3	213
Random I	–	–	6.84	55.8	1096	–
LUT-Q I	1408	125	3.81	26.5	1096	1911
Yeh MAP I	3057	263	5.33	40.9	1096	1911
Yeh soft	3054	263	4.03	28.3	1096	1911
MSR-FC U	2068	197	6.41	50.8	1096	1911
Ours	1188	112	4.24	29.7	1096	1911
Ours + functional	1605	175	5.98	46.3	1096	1911
Our mask, free weights	1216	119	3.01	46.2	71.7	1911
Random mask, free weights	–	–	2.85	44.8	71.7	–
MSR + learned init	534	45	0.0339	5.48	63.7	439

Tab. 10: Paired MNIST sums: measured computation. Target timing excludes the FLOP profiler and uses one process per GPU. *Search seconds include profiling overhead and shared scheduling. †Target GFLOPs are per task: the complete 64/256/1024 sweep on synthetic data, one prescribed support budget on real data; synthetic time covers only 1024. These are incomplete Torch operation counts, not FLOP/s. A dash indicates unavailable measurement.

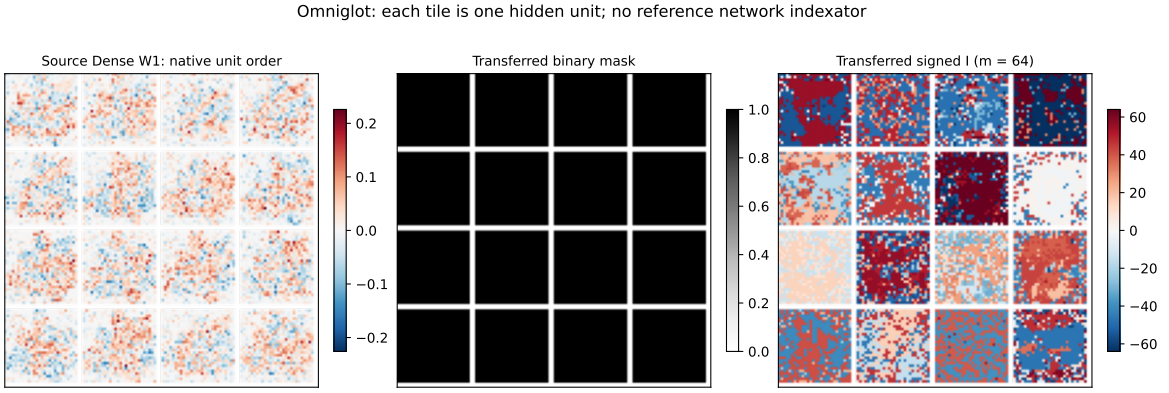


Fig. 6: Omniglot source weights, transferred mask and signed indexator, with one 28×28 tile per hidden unit. Hidden-unit orders are native to each model; numerical weights and arbitrary group labels have separate color scales.

References

[1] Jonathan Baxter. A model of inductive bias learning. *Journal of artificial intelligence research*, 12:149–198, 2000.

[2] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 10.1109/5.726791.

[3] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. 10.1207/s15516709cog1402_1.

[4] Siamak Ravanbakhsh, Jeff Schneider, and Barnabás Póczos. Equivariance through parameter-sharing. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2892–2901. PMLR, 2017. URL https://proceedings.mlr.press/v70/ravanbakhsh17a.html.

[5] Raymond A Yeh, Yuan-Ting Hu, Mark Hasegawa-Johnson, and Alexander Schwing. Equivariance discovery by learned parameter-sharing. In *International Conference on Artificial Intelligence and Statistics*, pages 1527–1545. PMLR, 2022.

[6] Allan Zhou, Tom Knowles, and Chelsea Finn. Meta-learning symmetries by reparameterization. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2007.02933.

[7] Steven J. Nowlan and Geoffrey E. Hinton. Adaptive soft weight tying using gaussian mixtures. In *Advances in Neural Information Processing Systems*, volume 4, pages 993–1000, 1991. URL https://proceedings.neurips.cc/paper/1991/hash/05f971b5ec196b8c65b75d2ef8267331-Abstract.html.

[8] Mehran Shakerinava, Motahareh Sohrabi, Siamak Ravanbakhsh, and Simon Lacoste-Julien. Weight-sharing regularization. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 4204–4212. PMLR, 2024. URL https://proceedings.mlr.press/v238/shakerinava24a.html.

[9] Andrei Atanov, Arsenii Ashukha, Kirill Struminsky, Dmitry Vetrov, and Max Welling. The deep weight prior. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1810.06943.

[10] Konstantin Schürholt, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Hyper-representations as generative models: Sampling unseen neural network weights. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2209.14733.

[11] Tianxiang Sun, Yunfan Shao, Xiaonan Li, Pengfei Liu, Hang Yan, Xipeng Qiu, and Xuanjing Huang. Learning sparse sharing architectures for multiple tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8936–8943, 2020. 10.1609/aaai.v34i05.6424. URL https://ojs.aaai.org/index.php/AAAI/article/view/6424.

[12] Rie Kubota Ando and Tong Zhang. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(61):1817–1853, 2005. URL https://jmlr.org/papers/v6/ando05a.html.

[13] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/f22e4747da1aa27e363d86d40ff442fe-Abstract.html.

[14] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016. URL https://arxiv.org/abs/1510.00149.

[15] Fabien Cardinaux, Stefan Uhlich, Kazuki Yoshiyama, Javier Alonso García, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Iteratively training look-up tables for network quantization. *arXiv preprint arXiv:1811.05355*, 2018. URL https://arxiv.org/abs/1811.05355.

[16] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1126–1135. PMLR, 2017. URL https://proceedings.mlr.press/v70/finn17a.html.

[17] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338, 2015. 10.1126/science.aab3050.